Perkalian Matriks Paralel: Analisis Komparatif POSIX Threads dan OpenMP

Laporan Tugas UTS Komputasi Pararel

Nama : Maulana Hasyim
NIM : (2322105061)
Prodi : Informatika

1. Pendahuluan

Perkalian matriks C = A × B adalah operasi inti di komputasi ilmiah, grafika, dan pembelajaran mesin. Untuk matriks N × N kompleksitas algoritma standar adalah $O(N^3)$: N^2 elemen hasil, masing-masing membutuhkan N perkalian-penjumlahan. Pada N = 1000 sudah diperlukan sekitar 2.0 miliar operasi floating point, sehingga operasi ini menjadi kandidat yang ideal untuk paralelisasi. Setiap elemen C[i][j] dapat dihitung secara independen, sehingga masalah ini bersifat *embarrassingly parallel* pada sisi komputasi, tetapi performanya sangat dipengaruhi oleh pola akses memori (cache locality) dan bandwidth memori.

Tugas ini mengimplementasikan perkalian matriks (tipe data double, ukuran minimal 1000 × 1000, inisialisasi srand(42)) dengan dua model pemrograman paralel memori-bersama: (1) POSIX Threads (Pthreads) yang eksplisit/manual, dan (2) OpenMP yang berbasis compiler directive.

Tujuan:

- Membuat baseline serial dan dua implementasi paralel yang menghasilkan hasil identik dan terverifikasi.
- Mengukur waktu eksekusi, speedup, dan efficiency untuk 1, 2, 4, dan 8 thread.
- Menganalisis efek urutan loop dan cache locality (i-j-k, i-k-j, transpose, tiling) terhadap performa.
- Membandingkan kompleksitas kode, kemudahan perawatan, dan kontrol yang ditawarkan Pthreads dibanding OpenMP, termasuk analisis bottleneck dengan Amdahl's Law.

Lingkungan pengujian: Intel(R) Xeon(R) Processor @ 2.10GHz; 1 core fisik / 1 CPU logis; cache L1d 48 KiB (1 instance), L2 2 MiB (1 instance), L3 260 MiB (1 instance); RAM 3.9 GB; Linux-6.18.44-fc-v185-x86_64-with-glibc2.39; gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0, opsi -O2.
Setiap konfigurasi diulang 3 kali dan dilaporkan sebagai rata-rata ± simpangan baku (waktu wall-clock, CLOCK_MONOTONIC).

2. Desain Implementasi

2.1 Algoritma Serial (Baseline)

Matriks disimpan row-major dalam array 1D (elemen (i,j) = M[i*N+j]). Nilai A dan B diisi rand()/RAND_MAX setelah srand(42) sehingga data identik pada setiap eksekusi. Empat varian kernel dibuat untuk menjawab tantangan optimasi memori:

Varian	Urutan loop / teknik	Pola akses memori
ijk	i, j, k; akumulasi ke variabel lokal	A[i][k] sekuensial, **B[k][j] melompat N elemen** (stride-N) pada loop dalam
ikj	i, k, j; C[i][j] += A[i][k]*B[k][j]	A[i][k] konstan di loop dalam; B dan C diakses **sekuensial** per baris
transpose	B ditranspos ke B^T (serial), lalu dot-product baris A × baris B^T	Kedua operand sekuensial; biaya tambahan $O(N^2)$ untuk transpos
tiled	i-k-j dengan blok 64 × 64 pada k dan j	Blok B (64×64×8 B = 32 KB) dipakai ulang selagi masih di cache

Kernel serial inti ditulis sekali dalam fungsi mm_rows(v, A, B, Bt, C, N, r0, r1) yang menghitung baris C[r0..r1-1]; baseline serial memanggilnya dengan r0=0, r1=N.

```
/* varian ikj (kernel baris r0..r1-1) */ for (int i = r0; i < r1; i++) {     for (int j = 0; j < N;
j++) C[i*N+j] = 0.0;     for (int k = 0; k < N; k++) {         double a = A[i*N+k];         for (int
j = 0; j < N; j++) C[i*N+j] += a * B[k*N+j];     }
}
```

Verifikasi kebenaran. Setiap program menjalankan *selftest* 10 × 10 sebelum pengukuran: (1) A[i][k]=i+1 dan B[k][j]=j+1, sehingga hasil manual C[i][j] = 10(i+1)(j+1) (mis. C[0][0]=10, C[2][4]=150, C[9][9]=1000); (2) A = matriks identitas sehingga C harus sama dengan B. Pada N besar, hasil setiap konfigurasi dibandingkan dengan matriks referensi memakai toleransi relatif 10^{-9}.

2.2 Strategi Paralelisasi Pthreads

Pembagian workload. Digunakan *domain decomposition* per baris matriks C. Thread ke-t menerima blok baris yang kontigu [r0, r1) dengan ukuran N/T; sisa N mod T disebar satu baris ke thread-thread pertama sehingga selisih beban antar thread maksimal satu baris. Pembagian berbasis baris dipilih karena (a) baris C bersifat independen, (b) tiap thread membaca baris A miliknya secara sekuensial, dan (c) thread menulis daerah memori yang berbeda sehingga risiko *false sharing* hanya ada di perbatasan blok.

Mekanisme sinkronisasi. Matriks A, B, dan B^T hanya dibaca (shared read-only), sedangkan setiap baris C hanya ditulis oleh tepat satu thread. Karena itu **tidak ada race condition dan tidak diperlukan mutex/semaphore**. Satu-satunya sinkronisasi adalah pthread_join di thread utama, yang berfungsi sebagai barrier akhir. Transpos B dikerjakan serial sebelum thread dibuat.

```
typedef struct { int v, N, r0, r1; const double *A,*B,*Bt; double *C; } task_t; static void *worker(void *arg) {
task_t *t = (task_t *)arg;
   mm_rows(t->v, t->A, t->B, t->Bt, t->C, t->N, t->r0, t->r1);     return NULL; } /* di
matmul(): */
int base = N / T, extra = N % T, row = 0; for (int t = 0; t < T;
t++) {     int cnt = base + (t < extra ? 1 : 0);
   tasks[t] = (task_t){v, N, row, row + cnt, A, B, Bt, C};     row += cnt;
   pthread_create(&th[t], NULL, worker, &tasks[t]);
}
for (int t = 0; t < T; t++) pthread_join(th[t], NULL);
```

2.3 Strategi Paralelisasi OpenMP

Pemilihan pragma dan clause. Paralelisasi dilakukan dengan #pragma omp parallel for pada loop terluar (indeks baris i), yang secara semantik setara dengan pembagian baris pada Pthreads. Clause schedule(static) dipilih karena beban setiap iterasi seragam, sehingga penjadwalan statis menghasilkan blok kontigu dengan overhead runtime terendah (penjadwalan dynamic/guided hanya menambah overhead tanpa manfaat). Jumlah thread diatur dengan omp_set_num_threads(T).
Barrier implisit di akhir *parallel for* menggantikan pthread_join.

Scope variabel. Memakai default(none) agar setiap variabel wajib dideklarasikan eksplisit (mencegah kesalahan scope yang diam-diam):

- **shared**: A, B (atau B^T), C, N. A dan B hanya dibaca; elemen C yang ditulis tidak pernah tumpang tindih antar thread.
- **private**: indeks loop i, j, k (dan ii, jj, kk pada varian tiled) serta akumulator sum; setiap thread punya salinan sendiri. Menjadikan sum shared adalah bug klasik yang menimbulkan race condition.

```
#pragma omp parallel for schedule(static) default(none) \         shared(A, B, C, N)
private(i, j, k) for (i = 0; i < N; i++) {
   for (j = 0; j < N; j++) C[i*N+j] = 0.0;     for (k = 0; k < N; k++) {         double a =
A[i*N+k];         for (j = 0; j < N; j++) C[i*N+j] += a * B[k*N+j];     }
}
```

3. Hasil Pengujian

3.1 Verifikasi Correctness

Seluruh konfigurasi (3 program × 4 varian × 4 variasi thread × 1 ukuran N) lulus selftest 10×10 dan menghasilkan matriks yang sama dengan referensi serial pada toleransi 10^{-9} (kolom *verified* = 1 pada seluruh 36 baris ringkasan). Bukti mentah ada di results/results.csv.

Karena varian ijk, ikj, transpose, dan tiled menjumlahkan suku-suku C[i][j] dengan urutan k yang sama (naik), perbedaan numerik antar varian praktis nol; toleransi tetap dipakai karena aritmetika floating point tidak asosiatif jika urutan penjumlahan berubah.

3.2 Waktu Eksekusi

Tabel 1 memuat waktu eksekusi (detik, rata-rata ± simpangan baku dari 3 ulangan) untuk N = 1000.

Program	Thread	ijk	ikj	transpose	tiled
serial	1	0.877 ± 0.022	0.636 ± 0.052	0.714 ± 0.017	0.647 ± 0.115
pthreads	1	0.903 ± 0.010	0.555 ± 0.018	0.715 ± 0.013	0.657 ± 0.019
pthreads	2	0.925 ± 0.051	0.672 ± 0.042	0.717 ± 0.011	0.664 ± 0.015
pthreads	4	0.908 ± 0.027	0.553 ± 0.020	0.741 ± 0.010	0.678 ± 0.030
pthreads	8	0.910 ± 0.050	0.558 ± 0.025	0.731 ± 0.023	0.705 ± 0.061
openmp	1	0.955 ± 0.049	0.616 ± 0.049	0.722 ± 0.020	0.758 ± 0.016
openmp	2	0.896 ± 0.036	0.601 ± 0.025	0.711 ± 0.022	0.758 ± 0.022
openmp	4	0.907 ± 0.031	0.645 ± 0.143	0.724 ± 0.021	0.773 ± 0.046
openmp	8	0.992 ± 0.066	0.670 ± 0.106	0.703 ± 0.019	0.769 ± 0.009

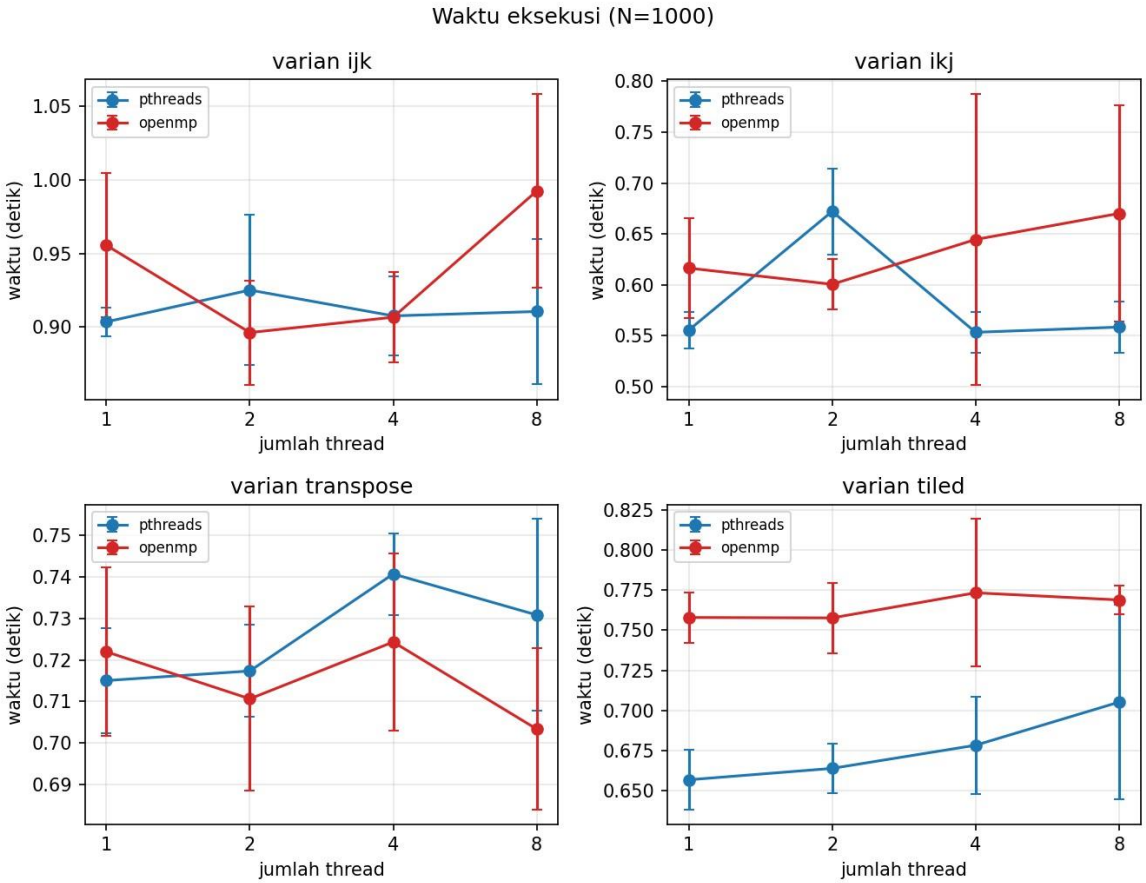
Tabel 1. Waktu eksekusi (detik), N = 1000.

3.3 Speedup dan Efficiency Speedup $S = T_{serial} / T_{paralel}$ dan efficiency $E = S / T$ dihitung terhadap waktu serial varian yang sama.

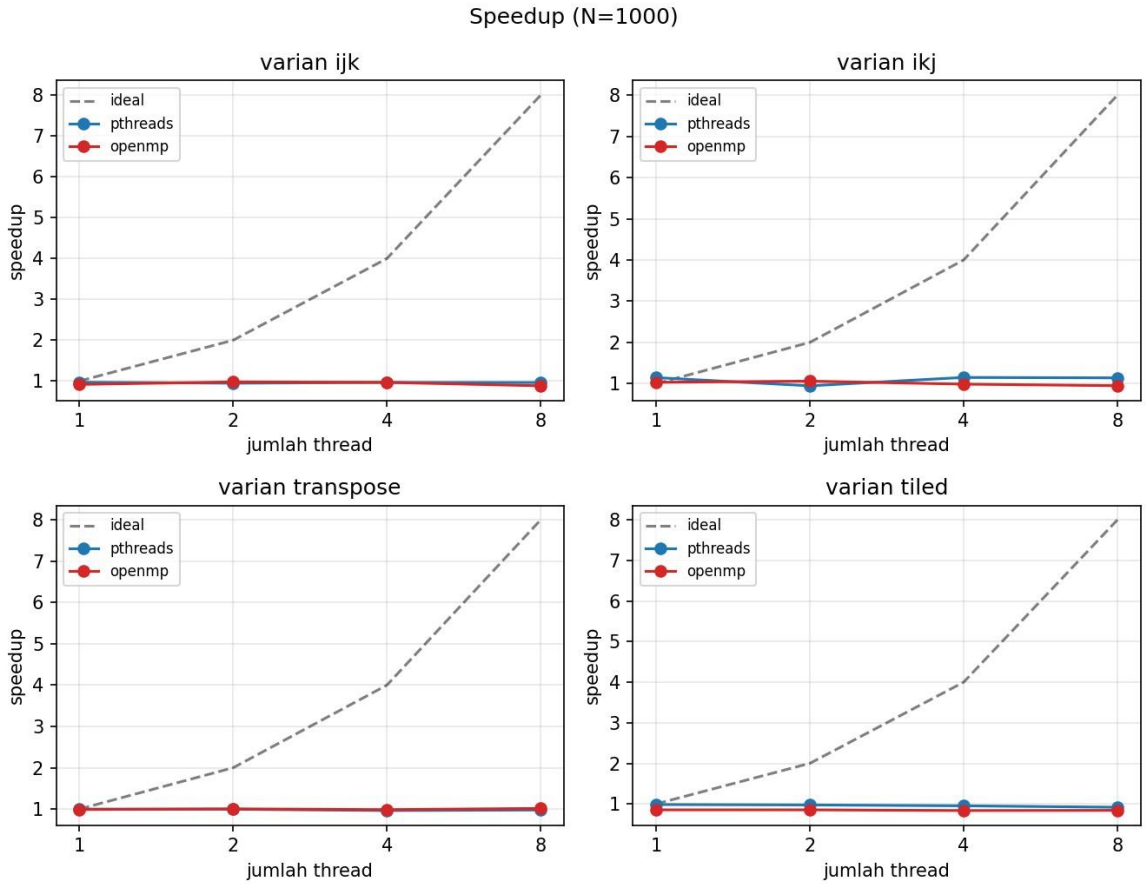
Tabel 3 merangkum varian tercepat pada baseline serial (ikj); grafik lengkap untuk keempat varian ditunjukkan pada Gambar 1-3.

Pendekatan	Thread	Waktu (s)	Speedup	Efficiency
pthreads	1	0.555	1.15	1.15
pthreads	2	0.672	0.95	0.47
pthreads	4	0.553	1.15	0.29
pthreads	8	0.558	1.14	0.14
openmp	1	0.616	1.03	1.03
openmp	2	0.601	1.06	0.53
openmp	4	0.645	0.99	0.25
openmp	8	0.670	0.95	0.12

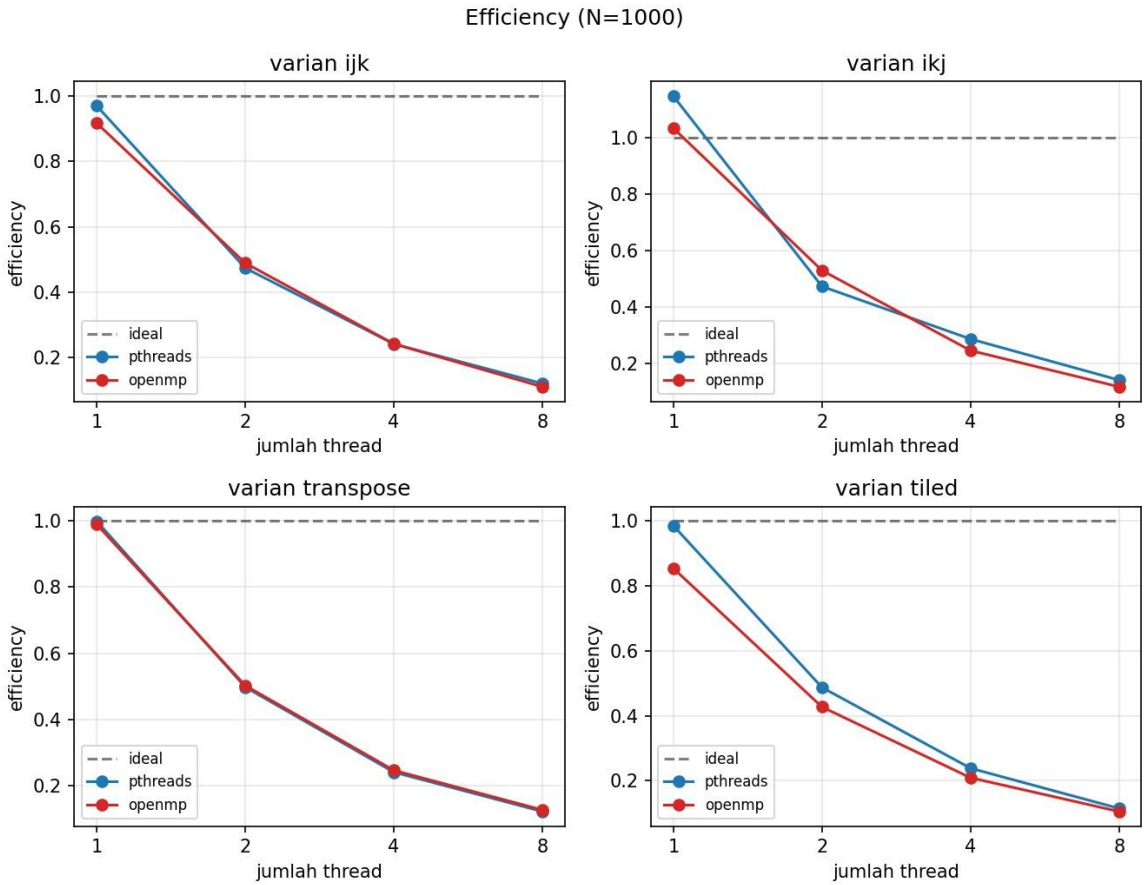
Tabel 3. Speedup dan efficiency varian ikj, N = 1000 (T_{serial} = 0.636 s).



Gambar 1. Waktu eksekusi vs jumlah thread (N = 1000). Error bar = 1 simpangan baku.



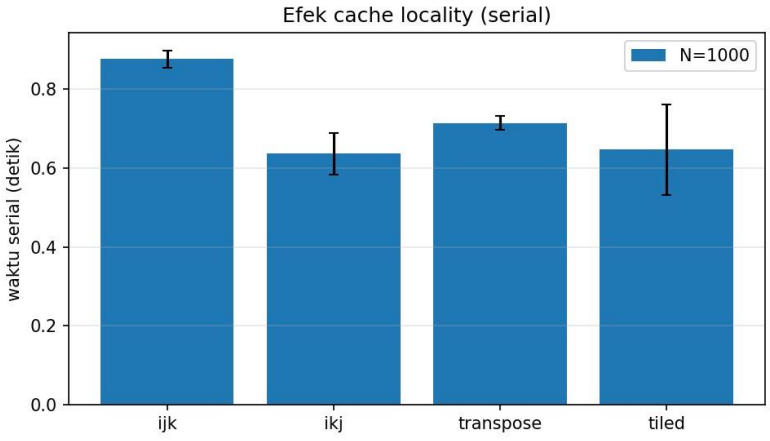
Gambar 2. Speedup vs jumlah thread (N = 1000); garis putus-putus = speedup ideal.



Gambar 3. Efficiency vs jumlah thread (N = 1000).

3.4 Efek Cache Locality (Baseline Serial)

Gambar 4 membandingkan waktu keempat varian pada eksekusi serial. Pada N = 1000, urutan ijk membutuhkan 1.38x waktu i-k-j; transpose 1.12x; tiled 1.02x dari waktu i-k-j.



Gambar 4. Waktu serial per varian kernel.

4. Analisis Kritis

4.1 Cache Locality dan Pola Akses Memori

Perbedaan antar varian serial berasal dari pola akses ke matriks B. Pada urutan i-j-k, loop terdalam (k) mengakses B[k][j] dengan lompatan N×8 = 8000 byte; setiap akses jatuh ke cache line yang berbeda (satu cache line 64 byte memuat 8 double), sehingga hanya 1 dari 8 elemen pada line yang termuat yang langsung dipakai, dan kolom B

harus dimuat ulang untuk setiap pasangan (i, j) jika tidak muat di cache. Pada urutan i-k-j, loop terdalam (j) membaca B[k][:] dan menulis C[i][:] secara sekuensial: seluruh byte tiap cache line terpakai, prefetcher hardware bekerja optimal, dan compiler dapat melakukan vektorisasi SIMD. Terukur pada N = 1000: i-j-k membutuhkan 1.38x waktu i-k-j, sedangkan varian transpose 1.23x lebih cepat daripada i-j-k. Rasio ini kemungkinan lebih kecil daripada pada CPU dengan cache kecil karena mesin uji memiliki L2 2 MiB (1 instance) dan L3 260 MiB (1 instance), dan prefetcher modern dapat menyembunyikan sebagian latensi akses ber-stride; pada mesin lain (atau N lebih besar) selisihnya dapat jauh lebih besar. Varian transpose membayar biaya $O(N^2)$ untuk transpos serial, tetapi dot-product sekuensial pada B^T menjadi jauh lebih ramah cache, sehingga biayanya cepat terbayar untuk N besar. Varian tiled menjaga blok B (32 KB) tetap di L1/L2 selama dipakai ulang oleh BS baris A; manfaatnya meningkat ketika working set (3 matriks × 8 MB pada N = 1000) jauh melampaui cache.

4.2 Perbandingan Kompleksitas Kode

Berkas	LOC total (tanpa komentar & baris kosong)	LOC lapisan paralel*	Keterangan
src/common.h	138	-	Kernel serial, timer, verifikasi, driver (dipakai bersama)
src/serial.c	9	-	Baseline
src/pthreads.c	32	29	6 referensi API pthread_*; struct argumen, worker, partisi manual, create/join
src/openmp.c	60	57	5 direktif #pragma omp; seluruh kernel ditulis ulang inline

*Baris di antara penanda PAR-BEGIN/PAR-END pada masing-masing berkas. Catatan kejujuran metodologis: pada kode ini Pthreads memakai ulang mm_rows dari common.h, sedangkan OpenMP menulis ulang kernel di dalam loop yang dianotasi pragma (pragma harus berada tepat di atas loop). Maka perbandingan LOC mentah tidak mencerminkan beban paralelisasinya. Ukuran yang lebih adil adalah *kode tambahan untuk paralelisasi*: OpenMP cukup menambah **satu baris pragma (+ 1 baris omp_set_num_threads)** pada kernel serial, sedangkan Pthreads menambah struct argumen, fungsi worker, perhitungan partisi, alokasi array thread, pembuatan, penggabungan, dan penanganan error: sekitar 29 baris (lapisan paralel) dan perubahan struktur program (loop harus diekstrak menjadi fungsi dengan parameter rentang baris).

Maintainability. Kode OpenMP yang hanya memakai pragma tetap menjadi program serial yang benar bila dikompilasi tanpa -fopenmp (pragma diabaikan), sehingga logika dapat di-debug secara serial.
(Pada implementasi ini pemanggilan omp_set_num_threads dan omp.h memerlukan guard #ifdef _OPENMP agar sifat tersebut berlaku.) Kode Pthreads mengikat logika komputasi dengan manajemen thread; setiap perubahan (mis. mengganti partisi baris menjadi blok 2D, atau menambah reduksi) menyentuh struktur data dan alur kontrol. Sebaliknya Pthreads menawarkan transparansi total: tidak ada perilaku tersembunyi dari runtime.

4.3 Bottleneck dan Amdahl's Law

Amdahl's Law menyatakan speedup maksimum $S(p) = 1 / (f + (1 - f)/p)$ dengan f adalah fraksi serial program. Bagian serial pada program ini adalah: inisialisasi data, transpos B (hanya varian transpose), pembuatan/penggabungan thread, dan *load imbalance* pada blok terakhir. Karena kernel inti mendominasi waktu ($O(N^3)$ dibanding $O(N^2)$ untuk bagian serial), f secara teori sangat kecil dan speedup ideal mendekati linear; batas praktisnya justru datang dari **bandwidth memori bersama** dan **cache bersama**, yang tidak tercakup model Amdahl.

Keterbatasan pengukuran: mesin yang menghasilkan data laporan ini hanya memiliki 1 core fisik, sehingga fraksi serial efektif tidak dapat diestimasi dari data (metrik Karp-Flatt memerlukan $p > 1$ pada core fisik yang nyata).

Sebagai gantinya Tabel 4 memberikan batas teoretis Amdahl untuk beberapa nilai f. Jalankan make bench pada mesin multi-core untuk mendapatkan estimasi empiris.

Fraksi serial f	S(2)	S(4)	S(8)	S(maks, p→∞)
1%	1.98	3.88	7.48	100
5%	1.90	3.48	5.93	20
10%	1.82	3.08	4.71	10
25%	1.60	2.29	2.91	4

Tabel 4. Batas speedup Amdahl untuk berbagai fraksi serial.

4.4 Pembahasan Anomali

Berikut temuan yang terdeteksi otomatis dari data pengukuran dan penjelasannya:

- **Oversubscription.** Pengujian sampai 8 thread dijalankan pada mesin dengan 1 CPU logis (1 core fisik). Thread yang melebihi jumlah CPU harus bergantian memakai core (context switch), sehingga speedup tidak dapat melampaui sekitar 1x dan efficiency turun hampir proporsional dengan 1/T.
- **Versi paralel dengan 1 thread tidak identik dengan serial.** Pada 2 konfigurasi, selisih waktu T=1 terhadap serial melebihi 10% (mis. pthreads/ikj, N=1000: speedup 1.15). Secara teori keduanya mengeksekusi kernel yang sama, jadi selisih ini berasal dari overhead pembuatan thread/runtime di satu sisi dan derau pengukuran / variasi frekuensi CPU serta efek alignment dan penempatan kode oleh compiler di sisi lain. Hal ini menjadi pengingat bahwa selisih di bawah ~10% pada pengukuran ini tidak boleh ditafsirkan sebagai keunggulan nyata.
- **Variansi antar-ulangan tinggi.** 3 konfigurasi memiliki simpangan baku > 10% dari rata-ratanya (terbesar: openmp/ikj, N=1000, T=4, 22%). Penyebab lazim: proses lain di sistem, penjadwalan thread oleh OS, throttling frekuensi, atau mesin virtual yang berbagi host.
- **Perbedaan Pthreads vs OpenMP.** Pada beberapa konfigurasi selisih waktu keduanya mencapai 17%, padahal strategi pembagian kerja serupa (blok baris kontigu). Kemungkinan penyebab: cara compiler meng-inline/mem-vektorisasi kernel (OpenMP memuat kernel langsung di dalam fungsi yang di-outline, Pthreads melalui mm_rows), serta derau pengukuran.
- **Tiling tidak lebih cepat dari i-k-j.** Pada N=1000, varian tiled berjalan 0.98x kecepatan i-k-j serial. Urutan i-k-j sudah hampir seluruhnya sekuensial dan di-vektorisasi otomatis oleh compiler pada -O2; tiling baru memberi keuntungan jelas bila ukuran data jauh melebihi kapasitas cache (mesin ini: L2 2 MiB (1 instance), L3 260 MiB (1 instance)) atau untuk N yang lebih besar, dan loop tambahan menambah overhead.

Perbandingan Pthreads vs OpenMP. Rata-rata geometrik rasio waktu Pthreads/OpenMP terhadap seluruh konfigurasi paralel adalah 0.947. Pthreads secara rata-rata lebih cepat sekitar 5%, namun selisih ini harus dibaca bersama simpangan baku pada Tabel 1.

4.5 Trade-off Kemudahan vs Kontrol

Aspek	Pthreads	OpenMP
Usaha menulis	Tinggi: partisi, struct argumen, create/join, error handling manual	Rendah: 1-2 direktif pada kode serial
Kontrol	Penuh: pembagian kerja, affinity, prioritas, pola sinkronisasi kustom, thread pool sendiri	Terbatas pada clause (schedule, collapse, proc_bind) dan perilaku runtime
Aspek	Pthreads	OpenMP
Risiko bug	Lebih tinggi (race, deadlock, kebocoran thread)	Lebih rendah, tetapi kesalahan scope (shared/private) sulit terlihat; default(none) membantu

Portabilitas	POSIX; Windows memerlukan lapisan tambahan	Standar bahasa lintas platform; bergantung dukungan compiler
Kemampuan lanjutan	Fleksibel untuk pola non-regular (pipeline, producer-consumer)	Tersedia task, reduction, SIMD, offload; paling cocok untuk loop data-parallel
Debugging/serial fallback	Tidak ada; kode selalu paralel	Dapat dikompilasi serial tanpa -fopenmp

Untuk perkalian matriks, beban kerja yang seragam dan loop teratur membuat OpenMP memberi hasil setara dengan usaha yang jauh lebih kecil. Kontrol ekstra Pthreads baru bernilai bila diperlukan pola yang tidak dapat diungkapkan sebagai *parallel for*.

5. Kesimpulan

Perkalian matriks ber-presisi ganda berhasil diimplementasikan dalam versi serial, Pthreads, dan OpenMP dengan empat varian akses memori; seluruh hasil terverifikasi benar terhadap selftest 10×10 dan referensi serial. Temuan utama:

- **Pola akses memori lebih menentukan daripada pilihan pustaka threading.** Pada serial N = 1000, i-k-j 1.38x lebih cepat dari i-j-k; rasio geometrik waktu Pthreads/OpenMP = 0.947 untuk kernel yang sama.
- **Skalabilitas dibatasi hardware.** Speedup tidak dapat melampaui jumlah core fisik (1 pada mesin uji) dan, pada mesin multi-core, dibatasi bandwidth/cache bersama; ini perlu dicek memakai estimasi fraksi serial efektif (Tabel 4).
- **Kompleksitas kode berbeda tajam:** paralelisasi OpenMP memerlukan kurang dari 5 baris tambahan, Pthreads sekitar 29 baris pada lapisan paralel.

Rekomendasi penggunaan. Gunakan **OpenMP** untuk komputasi data-parallel yang teratur (loop bersarang, reduksi, operasi matriks/array): waktu pengembangan singkat, kode mudah dirawat, dan performa setara. Gunakan **Pthreads** bila membutuhkan kontrol penuh: pola konkuren non-regular (pipeline, producer-consumer, server multi-thread), thread pool atau affinity kustom, atau lingkungan tanpa dukungan OpenMP.

Pembelajaran. (1) Optimasi serial (cache locality) harus dilakukan sebelum paralelisasi, karena ia memberi peningkatan yang tidak bergantung jumlah core. (2) Pengukuran yang valid memerlukan ulangan, wall-clock timer, dan pelaporan variansi; selisih kecil tidak bermakna. (3) Hasil paralel harus selalu diverifikasi terhadap acuan serial. (4) Keterbatasan perangkat keras (core fisik, bandwidth) harus dipertimbangkan saat menafsirkan speedup.

6. Daftar Pustaka

[1] Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computingcapabilities. *Proc. AFIPS Spring Joint Computer Conference*, 483-485.

[2] Butenhof, D. R. (1997). *Programming with POSIX Threads*. Addison-Wesley.

[3] Chapman, B., Jost, G., & van der Pas, R. (2008). *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press.

[4] Drepper, U. (2007). *What Every Programmer Should Know About Memory*. Red Hat, Inc.

[5] Hennessy, J. L., & Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach* (6th ed.). Morgan Kaufmann.

[6] Karp, A. H., & Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5), 539-543.

[7] OpenMP Architecture Review Board. (2021). *OpenMP Application Programming Interface, Version 5.2*. https://www.openmp.org

[8] Pacheco, P. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann.

[9] The Open Group. (2018). *POSIX.1-2017 (IEEE Std 1003.1): Threads*. https://pubs.opengroup.org/onlinepubs/9699919799/